Absolutely. **Thursday = Day 1: OSI + Transport Layer.**
And because your interview is next Thursday, we're going to treat today as **interview preparation**, not textbook study.

THURSDAY — CN DAY 1

Goal for today

By the end, you should be able to handle:

OSI
↓
TCP/IP
↓
7 layers + functions
↓
Protocols at each layer
↓
Encapsulation
↓
TCP vs UDP
↓
TCP 3-way handshake
↓
TCP termination
↓
Ports
↓
Flow control
↓
Congestion control
↓
Sockets

I've marked the **high-priority interview items with** ⭐.

Part 1 — OSI Model ⭐⭐⭐⭐⭐

1. Why does OSI exist?

Networking is complicated.
Instead of treating networking as one giant process, we divide it into **layers**, where each layer has a specific responsibility.
The OSI model has **7 layers**:

7 Application
6 Presentation
5 Session
4 Transport
3 Network
2 Data Link
1 Physical

The most useful mental model is:

L7 → What service?
L6 → What format?
L5 → What session?
L4 → Which process?
L3 → Which network?
L2 → Which device?
L1 → How physically?

---

2. Layer 1 — Physical

Job

Transmit **raw bits** over a physical medium.

101101001010

Examples:

- Copper cable
- Fiber
- Radio waves
- Electrical signals
- Optical signals

Data unit

Bits

Device

Hub is traditionally considered a Layer 1 device.

Interview answer

The Physical layer is responsible for transmitting raw bits as electrical, optical, or radio signals over the physical medium.

---

3. Layer 2 — Data Link

Job

Handle communication between devices on the **same local network**.

Responsibilities:

- Frames
- MAC addresses
- Error detection
- Local delivery

Examples:

- Ethernet
- Wi-Fi

Data unit

Frame

Address

MAC address

Device

Switch

Example:

PC A

MAC: AA:AA

↓

Switch

↓

PC B

MAC: BB:BB

The switch uses MAC addresses to decide where to forward the frame.

---

4. Layer 3 — Network

Job

Move packets between **different networks**.

Responsibilities:

- Logical addressing
- Routing
- Packet forwarding

Main protocol:
IP
Data unit
Packet
Address
IP address
Device
Router
Example:
Network A
↓
Router
↓
Network B
The router looks at the destination IP and decides where to forward the packet.

---

5. Layer 4 — Transport ⭐⭐⭐⭐⭐
This is one of the most important layers for SDE interviews.
Job
Provide **process-to-process communication**.
Responsibilities:

- Reliability
- Ordering
- Segmentation
- Retransmission
- Flow control
- Congestion control
- Port numbers

Main protocols:
TCP
UDP
Data unit
TCP:
Segment
UDP:
Datagram
Addressing
Port number
For example:
192.168.1.10:50000
The IP identifies the machine.
The port identifies the process/service.

---

6. Layer 5 — Session
Job
Manage communication sessions.
Establish
↓
Maintain
↓
Synchronize
↓
Terminate

Think:
"Are these two applications currently having a conversation, and how is that conversation maintained?"
Modern networking doesn't always have a clean standalone Session layer, so don't overthink protocol examples here.

---

7. Layer 6 — Presentation
Job
Handle **representation of data**.
Responsibilities:

- Encoding
- Encryption/decryption
- Compression/decompression
- Serialization concepts

Examples:
UTF-8
JSON
XML
TLS
JPEG
gzip
Think:
How should the data be represented?

---

8. Layer 7 — Application
Job
Provide network services used by applications.
Examples:
HTTP
HTTPS
DNS
FTP
SMTP
IMAP
POP3
SSH
DHCP
SNMP
Important:
Chrome is not an OSI Application-layer protocol.
HTTP is.

---

9. Complete Table
This is the table I want you to memorize.

Layer	Name	Main Responsibility	Address	Data Unit	Examples
7	Application	Network services	—	Data	HTTP, DNS, FTP
6	Presentation	Format/encryption/compression	—	Data	TLS, UTF-8
5	Session	Session management	—	Data	RPC/session concepts
4	Transport	Process-to-process delivery	**Port**	Segment/Datagram	TCP, UDP
3	Network	Routing between networks	**IP**	Packet	IP, ICMP
2	Data Link	Local delivery	**MAC**	Frame	Ethernet, Wi-Fi
1	Physical	Signal transmission	—	Bits	Fiber, copper, radio

The three addresses to remember:

L4 → Port
L3 → IP
L2 → MAC
That alone answers a surprising number of interview questions.

---

10. Encapsulation ⭐⭐⭐⭐⭐
Suppose you send:
Hello
from your computer.
At the sender:
Application
↓
"Hello"
↓
Transport
↓
TCP Segment
↓
Network
↓
IP Packet
↓
Data Link
↓
Ethernet Frame
↓
Physical
↓
Bits
This process is called:
Encapsulation
The receiver performs the reverse:
Bits
↓
Frame
↓
Packet
↓
Segment
↓
Data
That's:
Decapsulation

---

11. The Data Units
Very common interview question.
L7-L5 → Data
L4 → Segment / Datagram
L3 → Packet
L2 → Frame
L1 → Bits
Don't get hung up on the exact terminology around L5–L7. The important distinctions are L4/L3/L2/L1.

---

12. Protocol Mapping
If they ask:
"Which protocols belong to which OSI layers?"
Think:
L7
HTTP
DNS
FTP
SMTP
DHCP
SSH

L6
TLS
Encoding
Compression

L5
Session management / RPC concepts

L4
TCP
UDP

L3
IP
ICMP

L2
Ethernet
Wi-Fi
ARP*

L1
Physical media/signals
ARP is one of those awkward cases that doesn't map perfectly to a single OSI layer; don't let an interviewer force you into an overly rigid classification.

---

13. OSI vs TCP/IP ⭐ ⭐ ⭐ ⭐ ⭐
OSI TCP/IP

Application ──────┐
Presentation ──────┤
Session ──────┘ → Application

Transport ──────→ Transport

Network ──────→ Internet

Data Link ──────┐
Physical ──────┘ → Network Access
Interview answer

OSI is a seven-layer reference model, while TCP/IP is a practical protocol architecture used by the Internet. TCP/IP combines the OSI Application, Presentation, and Session layers into its Application layer, and generally combines Data Link and Physical into its Network Access layer.
That's enough.

Part 2 — TCP vs UDP ⭐ ⭐ ⭐ ⭐ ⭐
Now we move into the most important part of today's session.
TCP
TCP is:

- Connection-oriented
- Reliable
- Ordered
- Byte-stream based
- Has retransmission
- Has flow control
- Has congestion control

Think:
"I care about getting the data correctly and in order."

UDP
UDP is:

- Connectionless
- Datagram-based
- No guaranteed delivery
- No guaranteed ordering
- Lower overhead
- Faster to start transmitting

Think:
"I care more about low overhead/latency than TCP-style delivery guarantees."

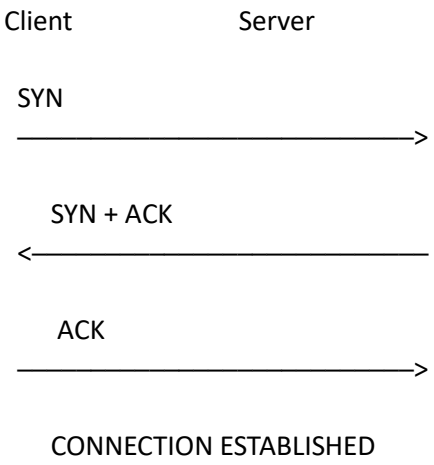
TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Yes	No guarantee
Ordering	Yes	No guarantee
Retransmission	Yes	Not built into UDP
Flow control	Yes	No TCP-style mechanism
Congestion control	Yes	No TCP-style mechanism
Data	Byte stream	Datagrams
Overhead	Higher	Lower
Handshake	Yes	No TCP handshake
Examples	HTTP/1.1, HTTPS, SSH	DNS, VoIP, gaming

One correction that is important in interviews:
UDP itself does not guarantee that it is "faster." It has less protocol overhead and no TCP-style connection establishment/reliability machinery, but actual application performance depends on the workload and implementation.

---

14. TCP 3-Way Handshake ⭐ ⭐ ⭐ ⭐ ⭐
This is **must-know**.
Client wants to establish TCP connection with server.



Step 1 — SYN

Client says:

"I want to establish a connection."

It sends:

SYN = 1

with an initial sequence number.

Step 2 — SYN-ACK

Server says:

"I received your request, and I also want to establish the connection."

It sends:

SYN = 1

ACK = 1

Step 3 — ACK

Client acknowledges the server's response.

ACK = 1

Now:

ESTABLISHED

Why THREE?

This is where interviewers may push you.

Don't just say:

"Because TCP uses three packets."

The real reason is that both sides need to establish and acknowledge their sequence-number state.

Conceptually:

Client → Server

"I can send."

Server → Client

"I can receive you, and I can send too."

Client → Server

"I received that."

Two messages wouldn't provide the same confirmation of both directions.

---

15. TCP Termination

TCP is full duplex, so each direction is closed independently.

Typical flow:



FIN
——————————————————>

ACK
<——————————————————

FIN
<——————————————————

ACK
——————————————————>

That's the famous **4-way termination**.
Why not 3?
Because TCP allows one side to stop sending while still receiving data from the other side.

---

16. Flow Control vs Congestion Control ⭐ ⭐ ⭐ ⭐ ⭐
This is a classic interview trap.
Flow Control
Protects the:
Receiver
Question:
"Can the receiver handle data this quickly?"
Controlled using concepts such as the receive window (rwnd).

Congestion Control
Protects the:
Network
Question:
"Can the network handle this much traffic?"
Uses concepts such as:
cwnd
Memorize:
Flow control → Receiver
Congestion control → Network
If you remember nothing else, remember that.

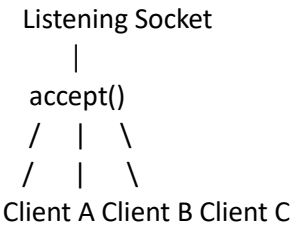
---

17. Ports ⭐ ⭐ ⭐ ⭐ ⭐
Suppose:
192.168.1.5
is your machine.
It may be running:
SSH → 22
HTTP → 80
HTTPS → 443
Postgres → 5432
So:
IP address → Machine
Port → Service/process endpoint
A TCP connection can be represented by:
Source IP
Source Port
Destination IP

Destination Port
Example:
192.168.1.5:53124
↓
142.250.x.x:443

---

18. Sockets ⭐⭐⭐⭐
A socket is a **communication endpoint**.
Server:
socket()
↓
bind()
↓
listen()
↓
accept()
Client:
socket()
↓
connect()
Important interview question:
What does accept() return?
A **new connected socket**.
The original listening socket remains available to accept additional clients.



Today's Core Revision
If I asked you these right now, you should be able to answer:
Q1
What are the seven OSI layers?
Q2
What does each layer do?
Q3
What is the difference between OSI and TCP/IP?
Q4
What is encapsulation?
Q5
What is the difference between a segment, packet, frame and bit?
Q6
TCP vs UDP?
Q7
Explain the TCP three-way handshake.
Q8
Why does TCP need a handshake?
Q9
Explain TCP connection termination.
Q10
Flow control vs congestion control?

Q11
What is a port?
Q12
What is a socket?
Q13
What does accept() do?

Friday — CN Day 2: Network + Data Link
Today we're moving down from Transport (L4) to Network (L3) and Data Link (L2).
The interview goal is not to become a network engineer. You need to be able to reason through:
IP → subnet → gateway → ARP → MAC → switch → router → NAT → destination

---

1. IP Addressing ⭐ ⭐ ⭐ ⭐ ⭐
An IP address identifies a network interface at the network layer and is used for logical addressing/routing.
IPv4:
192.168.1.10
IPv4 is 32 bits:
192 .168 .1 .10
8 bits 8 bits 8 bits 8 bits
Total:
8 × 4 = 32 bits

---

2. Public vs Private IP
Private IP
Used inside private networks.
The important IPv4 private ranges are:
10.0.0.0/8

172.16.0.0/12

192.168.0.0/16
Example:
192.168.1.10
This address is not directly routable across the public Internet.

Public IP
Globally routable address used to communicate across the Internet.
Typical home network:
Laptop
192.168.1.10
|
▼
Router
192.168.1.1
Public IP: X.X.X.X
|
▼
Internet

---

3. Subnet Mask ⭐ ⭐ ⭐ ⭐ ⭐
A subnet mask tells us which part of an IPv4 address represents the network and which part represents the host.
Example:
192.168.1.10

255.255.255.0
The mask is:
/24
Meaning:
24 network bits
8 host bits

---

4. CIDR ⭐ ⭐ ⭐ ⭐ ⭐
CIDR notation:
192.168.1.10/24
The /24 means:
First 24 bits are the network portion.
So:
Network bits = 24
Host bits = 32 - 24 = 8
Number of addresses:
2^8 = 256
Traditionally usable host addresses:
256 - 2 = 254
because:
192.168.1.0 → Network address
192.168.1.255 → Broadcast address
So usable:
192.168.1.1
...
192.168.1.254

---

5. Why Do We Subnet?
Suppose you have:
192.168.1.0/24
That's 256 addresses.
You could divide it into smaller networks.
For example:
192.168.1.0/25
Now:
128 addresses
You can have two /25 subnets:
192.168.1.0/25
192.168.1.128/25
The important idea:
Subnetting divides a larger network into smaller logical networks.

---

6. Default Gateway ⭐ ⭐ ⭐ ⭐ ⭐
Suppose your laptop is:
192.168.1.10
and the destination is:
8.8.8.8
Your laptop asks:
"Is 8.8.8.8 on my local subnet?"
No.
Therefore:
Laptop

192.168.1.10

|

▼

Default Gateway

192.168.1.1

|

▼

Internet

The default gateway is normally the router used to reach destinations outside the local network.

---

7. MAC vs IP ⭐⭐⭐⭐⭐

This is a classic interview question.

IP

Used for:

Logical addressing and routing

Example:

192.168.1.10

MAC

Used for:

Local/link-layer delivery

Example:

00:1A:2B:3C:4D:5E

Think:

IP → Where is the destination network?

MAC → Which local device/interface should receive this frame?

---

8. ARP ⭐⭐⭐⭐⭐

Now the interesting part.

Suppose your laptop wants to send something to:

192.168.1.1

It knows the IP address.

But Ethernet needs a MAC address.

So it uses:

ARP — Address Resolution Protocol

Conceptually:

Laptop:

Who has 192.168.1.1?

↓

Router:

192.168.1.1 is me.

My MAC is:

AA:BB:CC:DD:EE:FF

The laptop stores this mapping in its ARP cache.

192.168.1.1

↓

AA:BB:CC:DD:EE:FF

---

9. ARP Important Detail

ARP is used to resolve:
IPv4 address → MAC address
It does not resolve:
Domain → IP
That's DNS.
So:
google.com → IP
DNS

IP → MAC
ARP
Very common interview distinction.

---

10. Switch ⭐⭐⭐⭐⭐
A switch primarily operates at Layer 2.
It uses a:
MAC address table
Example:
MAC Address Port

AA:AA:AA:AA:AA:AA 1
BB:BB:BB:BB:BB:BB 2
CC:CC:CC:CC:CC:CC 3
Suppose:
PC A
|
Port 1
|
Switch
|
Port 2
|
PC B
The switch learns:
PC A MAC → Port 1
PC B MAC → Port 2
Then it can forward frames appropriately.

---

11. How Does a Switch Learn MAC Addresses?
This is a nice follow-up.
Suppose a frame arrives:
Source MAC = AA
Destination MAC = BB
The switch sees:
Source = AA
Incoming port = 1
So it learns:
AA → Port 1
Then it checks:
"Do I know where BB is?"
If yes:
Forward only to BB's port.
If no:

Flood the frame.

---

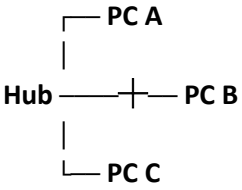
12. Switch vs Router ⭐⭐⭐⭐⭐

Switch	Router
Mainly L2	Mainly L3
Uses MAC	Uses IP
Forwards frames	Forwards packets
Connects devices/networks within local LAN context	Connects different networks
Uses MAC table	Uses routing table

Easy memory:
Switch → MAC
Router → IP

---

13. Hub vs Switch
A hub is much simpler.
If a hub receives data:



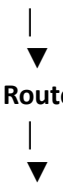
It basically sends the signal out through its other ports.
A switch is intelligent enough to use its MAC table to selectively forward frames.
Therefore:
Hub → Broadcast everything
Switch → Forward intelligently

---

14. Collision Domain vs Broadcast Domain
These can show up in basic networking questions.
Collision Domain
A part of the network where devices can potentially interfere with each other's transmissions.
Switches separate collision domains per port in normal switched Ethernet.
Broadcast Domain
A group of devices that receive the same Layer-2 broadcast.
Routers separate broadcast domains.
VLANs can also divide broadcast domains.

---

15. Router ⭐⭐⭐⭐⭐
A router connects different networks.
Example:
192.168.1.0/24



10.0.0.0/24
The router uses a routing table.
Example:
Destination Next Hop

192.168.1.0/24 Direct
10.0.0.0/24 192.168.1.1
0.0.0.0/0 ISP

---

16. Routing ⭐ ⭐ ⭐ ⭐ ⭐
Imagine:
Laptop
192.168.1.10

Destination:
8.8.8.8
The laptop checks its routing table.
It doesn't have a specific route for 8.8.8.8.
So it uses:
0.0.0.0/0
the default route.
That points toward the default gateway.

---

17. Longest Prefix Match
Suppose a router has:
10.0.0.0/8
10.20.0.0/16
10.20.30.0/24
Destination:
10.20.30.50
All three routes potentially match.
Which one wins?
10.20.30.0/24
because it has the longest matching prefix.
This is called:
Longest Prefix Match (LPM)
You don't need advanced routing-protocol internals for HPE, but understand this concept.

---

18. NAT ⭐ ⭐ ⭐ ⭐ ⭐
Your laptop has:
192.168.1.10
That's private.
It wants to access:
8.8.8.8
The Internet cannot directly route the private address.
Your router performs:
NAT — Network Address Translation
Conceptually:
192.168.1.10:50000
  |
  ▼
NAT
  |
  ▼
203.x.x.x:40001
  |
  ▼
Internet

The router keeps track of the translation.

---

19. Why NAT?
Historically, one major reason is:
IPv4 address conservation
Instead of every device requiring a public IPv4 address:
Laptop ┐
Phone ─┤
TV ─┤ → One public IP
Tablet ┘
NAT/PAT allows many private devices to share a public IPv4 address.

---

20. ICMP ⭐⭐⭐⭐
ICMP is used for network control/error reporting and diagnostics.
It isn't TCP.
It isn't UDP.
A famous example:
ping
Ping uses:
ICMP Echo Request
ICMP Echo Reply

---

21. Traceroute
Traceroute helps identify the path packets take through routers.
Conceptually:
Your PC
↓
Router 1
↓
Router 2
↓
Router 3
↓
Destination
It manipulates the IP TTL so intermediate routers eventually return messages indicating that the TTL expired.
Thus you can see the hops.

---

22. DHCP ⭐⭐⭐⭐
DHCP automatically provides network configuration.
Usually:
IP address
Subnet mask
Default gateway
DNS server
The classic sequence is:
DORA
D → Discover
O → Offer
R → Request
A → ACK
Step 1
Client broadcasts:

"Is there a DHCP server?"
Step 2
Server:
"I can offer you this IP."
Step 3
Client:
"I want that IP."
Step 4
Server:
"Okay, it's yours."

---

23. The Full Local-to-Internet Flow
This is the most important scenario from today's material.
Suppose:
Laptop:
192.168.1.10

Gateway:
192.168.1.1

Destination:
8.8.8.8
Laptop checks:
Is 8.8.8.8 in my subnet?
No.
Therefore:
Send to default gateway.
But to create the Ethernet frame, it needs the gateway's MAC.
So:
ARP
↓
192.168.1.1 → Gateway MAC
Then:
Laptop
|
| Ethernet Frame
▼
Switch
|
▼
Router
|
| NAT
▼
Internet
|
▼
Destination
At every router:
Destination IP
↓
Routing Table
↓

Next Hop
The Layer-2 frame is rebuilt for each local link, while the IP packet is routed across the path.
That last sentence is a very good interview-level insight.

---

24. One Crucial Distinction
Suppose:
Laptop → Router → Internet → Server
Does the laptop send a frame directly to the server's MAC?
No.
The laptop's Layer-2 destination is typically the MAC of the next hop, usually its default gateway.
The IP destination remains the final destination IP.
Conceptually:
L2:
Laptop MAC → Gateway MAC

L3:
Laptop IP → Server IP
At the next hop, the Layer-2 addresses change again.
This is one of the best ways to understand why MAC and IP serve different purposes.

---

25. Interview Questions You Should Now Be Able to Answer
Try these without looking back:
Basic

1. **What is an IP address?**
2. **Public vs private IP?**
3. **What is a subnet mask?**
4. **What does /24 mean?**
5. **What is a default gateway?**

L2/L3

6. **MAC vs IP?**
7. **What is ARP?**
8. **How does a switch forward a frame?**
9. **Switch vs router?**
10. **Hub vs switch?**
11. **Collision domain vs broadcast domain?**

Routing

12. **What is a routing table?**
13. **What is a default route?**
14. **What is longest-prefix matching?**
15. **Why does traffic go through a gateway?**

NAT

16. **What is NAT?**
17. **Why is NAT used?**
18. **What is PAT?**

Diagnostics

19. **What protocol does ping use?**
20. **How does traceroute work conceptually?**

DHCP

21. **What is DHCP?**
22. **What is DORA?**

The Mental Model for Today
If you remember only this:

APPLICATION
|
TCP / UDP
|
IP ADDRESS
|
ARP
|
MAC ADDRESS
|
SWITCH
|
ROUTER
|
NAT
|
INTERNET

And:
L4 → Port
L3 → IP
L2 → MAC
you've got the backbone of today's material.

One thing I want you to notice
This is why OSI shouldn't be memorized as seven isolated definitions.
A real packet gives you the complete chain:
Application data
↓
TCP segment
↓
IP packet
↓
Ethernet frame
↓
Bits
And when the destination is outside your LAN:
IP → determine destination network
MAC → reach the next local hop
Router → choose next network
NAT → potentially translate address/port

Day 3 — Application Layer + Real-World Networking
This is **high priority for your HPE SDE interview**, because the question list you found explicitly mentions **HTTP, FTP, DNS, SMTP, data flow, email, and basic networking problem-solving**.
Today we'll connect the pieces rather than memorize protocols independently.

---

1. Application Layer — What Does It Actually Do?
The Application layer provides network services that applications use to communicate.
Examples:
HTTP → Web
DNS → Name resolution
FTP → File transfer

SMTP → Send email
IMAP → Retrieve/synchronize email
POP3 → Retrieve email
SSH → Remote access
DHCP → Network configuration
A common mistake:
"Chrome is an Application-layer protocol."
No.
HTTP is the protocol. Chrome is the application using HTTP.

---

2. HTTP ⭐⭐⭐⭐⭐
HTTP = **HyperText Transfer Protocol**
It follows a request-response model:
Client Server

HTTP Request
——————————————————>

HTTP Response
<——————————————————
Example:
GET /index.html HTTP/1.1
Host: example.com
Server:
HTTP/1.1 200 OK

<html>...</html>

---

3. HTTP Request
An HTTP request contains things such as:
Method
Path
Headers
Body
Example:
POST /users HTTP/1.1
Host: example.com
Content-Type: application/json

{
"name": "Manthan"
}

---

4. HTTP Methods ⭐⭐⭐⭐⭐
Know these:

Method	Typical purpose
GET	Retrieve
POST	Create/submit
PUT	Replace/update
PATCH	Partial update
DELETE	Delete

Method	Typical purpose
HEAD	Headers only
OPTIONS	Supported operations

For REST:
GET /users/10
POST /users
PUT /users/10
PATCH /users/10
DELETE /users/10

---

5. HTTP Status Codes ⭐⭐⭐⭐⭐
Remember the categories:
1xx → Informational
2xx → Success
3xx → Redirection
4xx → Client error
5xx → Server error
Important ones:
200 → OK
201 → Created
204 → No Content

301 → Moved Permanently
302 → Found / Redirect

400 → Bad Request
401 → Unauthorized
403 → Forbidden
404 → Not Found
405 → Method Not Allowed
429 → Too Many Requests

500 → Internal Server Error
502 → Bad Gateway
503 → Service Unavailable
504 → Gateway Timeout
Classic trap
401 vs 403
401 → Authentication problem
403 → Server understood request but refuses access

---

6. HTTP Is Stateless
HTTP itself does not inherently remember previous requests.
For example:
Request 1:
GET /profile

Request 2:
GET /profile

Request 3:
GET /profile

Each request is independently understood at the HTTP protocol level.
So how do websites maintain login state?
Using mechanisms such as:
Cookies
Sessions
Tokens
JWT

---

7. Cookies
A server can send:
Set-Cookie: sessionId=abc123
The browser stores it.
Later:
Cookie: sessionId=abc123
So:
Server
↓
Set-Cookie
↓
Browser
↓
Cookie
↓
Next Request

---

8. HTTP vs HTTPS ⭐⭐⭐⭐⭐
HTTPS is essentially:
HTTP + TLS
TLS provides:

- Confidentiality
- Integrity
- Server authentication

Conceptually:
HTTP
↓
TLS
↓
TCP
↓
IP
For HTTP/3, the transport is different:
HTTP/3
↓
QUIC
↓
UDP

---

9. FTP
FTP = **File Transfer Protocol**
Used for transferring files.
Traditional FTP uses:
TCP
Common ports:

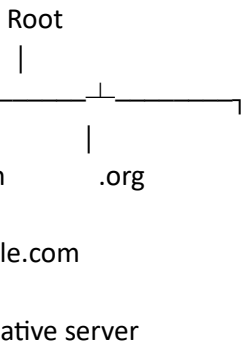
21 → Control
20 → Traditional active-mode data connection
Important:
FTP is not encrypted by default.
Secure alternatives include:
SFTP
SCP
FTPS
Don't confuse:
FTP ≠ SFTP
SFTP is an SSH-based file-transfer protocol.

---

10. DNS ⭐⭐⭐⭐⭐
DNS = **Domain Name System**
Its basic purpose:
Domain name
↓
IP address
Example:
google.com
↓
142.x.x.x
But the actual resolution process is important.

---

11. DNS Hierarchy
Think:



A resolver may need to work through:
Root Server
↓
TLD Server
↓
Authoritative Server

---

12. Recursive Resolver
Your computer usually doesn't directly ask the entire DNS hierarchy itself.
Instead:
Your Computer
|
▼
Recursive DNS Resolver
|
├— Root
├— TLD

└── Authoritative

The resolver does the work and returns the result.

---

13. DNS Cache ⭐ ⭐ ⭐ ⭐ ⭐

Suppose you already resolved:

google.com → 142.x.x.x

You don't want to perform the entire lookup every time.

So DNS results are cached.

DNS Query

↓

Cache?

┌─┴─┐

Yes No

| |

IP DNS lookup

---

14. TTL

DNS records have a **TTL — Time To Live**.

Example:

google.com

TTL = 300 seconds

The cached record can generally be considered valid for that period.

After expiration, another lookup may be needed.

This is why DNS changes aren't necessarily visible everywhere immediately.

---

15. Important DNS Records

You should know these:

Record	Purpose
A	Domain → IPv4
AAAA	Domain → IPv6
CNAME	Alias
MX	Mail server
NS	Name server
TXT	Text/metadata
PTR	Reverse DNS

The **MX record** becomes especially important when we discuss email.

---

16. DNS Transport ⭐ ⭐ ⭐ ⭐

DNS commonly uses:

UDP port 53

But DNS is **not UDP-only**.

TCP can also be used, including for situations such as larger responses and zone transfers.

Interview answer:

DNS commonly uses UDP on port 53 because it has low overhead, but TCP is also used when required.

---

17. DNS vs DHCP

Easy confusion.

DNS

Name → IP

DHCP

Give device network configuration

DHCP can provide:
IP address
Subnet mask
Gateway
DNS server
So:
DNS → "What IP belongs to example.com?"

DHCP → "Here's your IP configuration."

---

18. SMTP ⭐ ⭐ ⭐ ⭐ ⭐
SMTP = **Simple Mail Transfer Protocol**
Its primary job:
Send/relay email.
Think:
Sender
↓
SMTP
↓
Mail Server
↓
SMTP
↓
Recipient Mail Server
SMTP is not the normal protocol your email client uses to read its mailbox.
That's where IMAP/POP3 come in.

---

19. IMAP vs POP3 ⭐ ⭐ ⭐ ⭐
IMAP
Keeps email synchronized with the server.

```
        Server
       /     \
    Phone    Laptop
```

Both devices can see the same mailbox state.
POP3
Traditionally focuses on downloading mail to a client.
So:
SMTP → Send
IMAP → Synchronize/read mailbox
POP3 → Download/retrieve
Memorize this trio.

---

20. How Email Actually Works ⭐ ⭐ ⭐ ⭐ ⭐
This is explicitly one of the interview scenarios from the question list.
Suppose:
Alice@gmail.com
↓
sends
↓
Bob@example.com
Step 1 — Alice sends the email
Her mail client communicates with her mail service.

SMTP is involved in sending/relaying the message.

Alice
↓
SMTP
↓
Alice's Mail Server

Step 2 — Find Bob's mail server

Alice's mail server needs to know:
"Which server handles email for example.com?"
It queries DNS for the:
MX record
Example:

example.com
↓
DNS
↓
MX
↓
mail.example.com

Step 3 — SMTP delivery

The sending mail server communicates with Bob's mail server using SMTP.

Alice
↓
Mail Server A
|
| DNS MX lookup
▼
Mail Server B
↓
Bob's mailbox

Step 4 — Bob reads the email

Bob's email client retrieves/synchronizes the mailbox using:
IMAP
or potentially:
POP3
Complete flow:

Alice
|
| SMTP
▼
Alice's Mail Server
|
| DNS → MX
▼
Bob's Mail Server
|
| stores mail
▼
Bob's Email Client
|

├— IMAP
└— POP3

Interview-ready answer

SMTP is used to send and relay email between mail servers. DNS MX records identify the mail server responsible for the recipient's domain. The receiving server stores the message, and the recipient retrieves or synchronizes it using protocols such as IMAP or POP3.

That's a **very strong 30-second answer**.

---

21. SSH

SSH = Secure Shell.

Used for:

- Remote login
- Secure command execution
- Secure file transfer mechanisms
- Tunneling

Default port:

22

Conceptually:

Your Computer
|
| encrypted SSH connection
▼
Remote Server

---

22. WebSocket ⭐ ⭐ ⭐ ⭐

HTTP traditionally follows:

Client → Request

Server → Response

WebSocket provides a persistent, bidirectional communication channel.

Client
⇄
Server

Useful for:

- Chat
- Live dashboards
- Multiplayer games
- Real-time applications

---

23. HTTP vs WebSocket

HTTP	WebSocket
Request/response	Persistent connection
Client generally initiates requests	Both sides can send
Good for REST APIs	Good for real-time communication
Stateless application semantics	Stateful connection

Don't say:

"WebSocket is faster than HTTP."

That's too simplistic.

The important difference is **communication model**, not simply speed.

---

24. HTTP/1.1 vs HTTP/2 vs HTTP/3 ⭐ ⭐ ⭐ ⭐

This is worth knowing at a high level.

HTTP/1.1

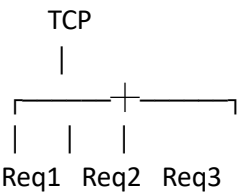
HTTP
↓
TCP

Uses persistent TCP connections, but requests can suffer from head-of-line issues at the HTTP level.

HTTP/2

Major improvements:

- Binary framing
- Multiplexing
- Header compression
- Multiple streams over a TCP connection



HTTP/3

Uses:

HTTP/3
↓
QUIC
↓
UDP

QUIC provides:

- Reliable delivery
- Encryption
- Multiplexed streams
- Faster connection establishment

You do **not** need QUIC implementation details for HPE.

---

25. The Most Important Scenario: "What Happens When You Type a URL?"

This is the question that ties together almost everything we've studied.

Suppose you type:

https://example.com

Step 1 — Browser

Browser checks things such as its cache and existing connections.

Step 2 — DNS

Need to resolve:

example.com
↓
IP address

DNS lookup happens.

Step 3 — Routing

Your machine determines how to reach that IP.

If it's outside your local subnet:

Default Gateway

is used.

ARP may resolve the gateway's MAC address.

Step 4 — TCP
For traditional HTTPS:
3-way handshake
SYN
↓
SYN-ACK
↓
ACK

Step 5 — TLS
HTTPS establishes a secure TLS session.
The server's certificate is validated and cryptographic keys are established.

Step 6 — HTTP
Browser sends:
GET / HTTP/1.1
Host: example.com

Step 7 — Server
The request might pass through:
CDN
↓
Load Balancer
↓
Reverse Proxy
↓
Application Server

Step 8 — Response
Server sends:
HTTP 200 OK
with the content.

Complete picture
Browser
|
▼
DNS
|
▼
IP Address
|
▼
ARP / Gateway
|
▼
TCP
|
▼
TLS
|
▼
HTTP

|
▼
Server
|
▼
HTTP Response
If you can explain this clearly, you are demonstrating understanding of a huge chunk of CN rather than just memorizing definitions.

---

26. Basic Networking Troubleshooting ⭐⭐⭐⭐⭐
The interview list explicitly mentions basic problem-solving, so let's build a systematic approach.
Suppose:
"My computer cannot access a website."
Don't randomly say:
"Maybe DNS."
Work from bottom to top.
Physical
↓
Link
↓
IP
↓
Routing
↓
DNS
↓
TCP
↓
TLS
↓
HTTP

Case 1 — Can ping IP, but domain doesn't work
Try:
ping 8.8.8.8
If that works but:
ping google.com
fails, suspect:
DNS
Use:
nslookup google.com
or:
dig google.com

Case 2 — DNS works, but website doesn't
Suppose:
DNS → works
but:
https://example.com → doesn't work
Now investigate:
TCP port 443
↓
Firewall

↓
TLS
↓
HTTP
You can test connectivity with:
nc -vz example.com 443

Case 3 — Server is running but unreachable
Check:
ss -tulpn
Ask:
Is the process running?
↓
Is it listening?
↓
Correct port?
↓
Firewall?
↓
Routing?
↓
DNS?

Case 4 — Two machines on same LAN can't communicate
Think:
Physical connection
↓
Interface
↓
IP configuration
↓
Subnet
↓
ARP
↓
Firewall
Useful:
ip addr
ip route
ip neigh

---

27. Your Friday/Saturday Interview Checklist
Before we leave today's material, you should be able to answer:
Application

- What is HTTP?
- HTTP request vs response?
- GET vs POST vs PUT vs PATCH?
- Important HTTP status codes?
- Why is HTTP stateless?
- What are cookies?
- HTTP vs HTTPS?
- What is FTP?
- FTP vs SFTP?

- What is DNS?
- How does DNS resolution work?
- What is a DNS resolver?
- What is DNS caching?
- What is TTL?
- A vs AAAA vs CNAME vs MX?
- Why does DNS use UDP?
- When can DNS use TCP?
- What is SMTP?
- SMTP vs IMAP vs POP3?
- How does email work?
- What is SSH?
- What is WebSocket?
- HTTP/1.1 vs HTTP/2?
- HTTP/2 vs HTTP/3?

Scenarios

- What happens when you type a URL?
- What happens when DNS fails?
- Ping works but website doesn't — what do you investigate?
- Two machines cannot communicate — how do you troubleshoot?
- Server is running but clients can't connect — what do you check?
- How does an email reach another domain?

Next Topic — Modern Web Networking: HTTP/1.1 vs HTTP/2 vs HTTP/3 + QUIC

This is a good next topic because you've already covered HTTP basics. Now we answer the natural interviewer follow-up:

"HTTP/1.1, HTTP/2 and HTTP/3 — what's the difference?"

This is high-value but not huge. For HPE SDE, understand the architecture and trade-offs; don't dive into protocol implementation internals.

---

1. First: Why did HTTP evolve?

The basic model stayed:

Client
|
| Request
▼
Server
|
| Response
▼
Client

But websites became much more complex.

A modern page may require:

HTML
CSS
JavaScript
Images
Fonts
API calls
Videos

So the browser may need to make many requests.

HTTP/1.1 had limitations that became increasingly noticeable.

---

2. HTTP/1.1

HTTP/1.1 uses:

HTTP

↓

TCP

↓

IP

It introduced persistent connections, meaning you don't necessarily need a new TCP connection for every HTTP request.

Example:

TCP Connection

|

├— Request 1 → Response 1

├— Request 2 → Response 2

└— Request 3 → Response 3

That's better than opening a new TCP connection every time.

---

3. The Problem: Head-of-Line Blocking

This is an important concept.

Imagine:

Request 1

Request 2

Request 3

Suppose Request 1 is delayed.

In HTTP/1.1 pipelining, later responses can be held up by the earlier one.

Conceptually:

Request 1 ——————————X

Request 2 ————————┐

Request 3 ————————┤

|

▼

Waiting...

This is head-of-line blocking.

The important idea:

A problem with an earlier request can delay subsequent responses.

---

4. How Did HTTP/1.1 Deal With This?

Browsers could open multiple TCP connections.

Instead of:

One TCP connection

├— Request 1

├— Request 2

└— Request 3

they could use:

TCP 1 → Requests

TCP 2 → Requests

TCP 3 → Requests

But this creates additional overhead:

- **More TCP connections**
- **More resources**
- **More handshakes**
- **More congestion-control state**

So HTTP/2 improved the situation.

---

5. HTTP/2 ⭐⭐⭐⭐⭐

HTTP/2 still commonly uses:

HTTP/2

↓

TCP

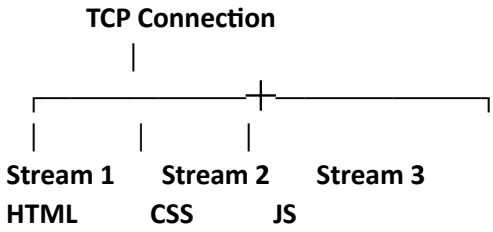
↓

IP

The major change is:

Multiplexing

Instead of treating each request as a separate connection, multiple streams can share one TCP connection.



This is a major interview concept.

---

6. Multiplexing

Suppose we have:

A

B

C

HTTP/1.1 might effectively process them with separate connections or sequential behavior.

HTTP/2 can interleave pieces:

A1

B1

C1

A2

C2

B2

A3

...

All over one TCP connection.

So:

Multiple HTTP streams can be multiplexed over a single TCP connection.

---

7. Binary Framing

HTTP/1.x is fundamentally text-oriented.

HTTP/2 introduces binary framing.

Instead of treating the connection simply as a stream of textual HTTP messages, HTTP/2 breaks communication into binary frames.

Conceptually:

HTTP/2

↓

Frames

↓

Streams

You don't need to memorize the frame format.

Just know:

HTTP/2 uses binary framing and multiplexed streams.

---

8. Header Compression

HTTP requests often contain repetitive headers:

Host
User-Agent
Cookie
Accept
Authorization
...

Sending the same headers repeatedly wastes bandwidth.

HTTP/2 uses:

HPACK

for header compression.

For HPE:

Know the concept; don't memorize HPACK internals.

---

9. HTTP/2 Summary

HTTP/2 gives us:

HTTP/2
│
├── Multiplexing
├── Binary framing
├── Header compression
└── One TCP connection can carry multiple streams

The major benefit:

Better utilization of a connection and reduced application-level head-of-line blocking.

But there's a catch.

---

10. HTTP/2 Still Uses TCP

This is the important transition.

Suppose HTTP/2 has:

Stream A
Stream B
Stream C
all over:
TCP

Now imagine a TCP packet is lost.

TCP guarantees ordered byte-stream delivery.

So TCP may need to wait for retransmission before delivering later bytes to the application.

Conceptually:

Stream A — X
Stream B — ?
Stream C — ?

TCP
│
"I need that
missing data."

So TCP-level head-of-line blocking can still affect HTTP/2.

This leads us toward HTTP/3.

---

11. HTTP/3 ⭐ ⭐ ⭐ ⭐ ⭐

HTTP/3 changes the transport stack:

HTTP/3
↓
QUIC
↓
UDP
↓
IP

This is the key fact to remember.

HTTP/1.1
HTTP
↓
TCP

HTTP/2
HTTP/2
↓
TCP

HTTP/3
HTTP/3
↓
QUIC
↓
UDP

---

12. Wait — Isn't UDP Unreliable?

Yes.

And this is where people often give a bad interview answer.

They say:

"HTTP/3 uses UDP, so HTTP/3 is unreliable."

Wrong.

UDP itself provides very little:

UDP
├— No TCP-style reliability
├— No ordering guarantee
└— No TCP-style congestion control

But QUIC implements the transport features needed by applications.

So:

UDP
↓
QUIC
↓
Reliable encrypted multiplexed transport

Think of UDP as the underlying datagram mechanism, while QUIC builds a modern transport protocol on top of it.

---

13. What Is QUIC?

QUIC is a modern transport protocol.

It provides things such as:

- **Reliable delivery**
- **Stream multiplexing**
- **Encryption**
- **Congestion control**
- **Connection migration**

And it runs over UDP.

The important mental model:
Traditional:

HTTP
↓
TLS
↓
TCP
↓
IP

HTTP/3:

HTTP/3
↓
QUIC
↓
UDP
↓
IP

---

14. Why QUIC Helps
One major advantage is that QUIC supports independent streams.
Suppose:
Stream A → packet lost
Stream B → packets available
Stream C → packets available
A loss affecting Stream A doesn't necessarily block delivery of data from B and C in the same way TCP's single ordered byte stream can.
Conceptually:
Stream A — X — waiting
Stream B ——————————→
Stream C ——————————→
That's a major reason QUIC is useful for HTTP/3.

---

15. QUIC Also Integrates TLS
With traditional HTTPS:
TCP handshake
↓
TLS handshake
↓
HTTP
QUIC integrates TLS 1.3 into the protocol architecture.
So connection establishment can be more efficient.
You don't need to memorize the exact number of packets exchanged.
For interview purposes:
QUIC integrates TLS and transport functionality, reducing connection-establishment overhead compared with the traditional TCP + TLS stack.

---

16. Connection Migration
This is a nice HTTP/3/QUIC feature.
Suppose you're using your phone:

Wi-Fi
↓
Mobile Data
Your network address can change.
Traditional TCP connections are strongly tied to the connection's endpoint addresses.
QUIC uses a connection ID, allowing a connection to survive certain network-path changes.
Conceptually:
Laptop/Phone
| Wi-Fi
▼
Server

↓ switch network

Mobile Data
|
▼
Server
The QUIC connection can potentially continue.
This is a nice-to-know, not a must-memorize HPE question.

---

17. The Big Comparison

Feature	HTTP/1.1	HTTP/2	HTTP/3
Transport	TCP	TCP	QUIC/UDP
Persistent connection	Yes	Yes	Yes
Multiplexing	Limited	Yes	Yes
Binary framing	No	Yes	Yes
Header compression	No standard built-in equivalent	HPACK	QPACK
Encryption	HTTPS via TLS	HTTPS via TLS	TLS integrated with QUIC
TCP HOL blocking	Yes	Yes	Avoided at transport-stream level
Connection migration	No	No	Yes, via QUIC
Main goal	Basic web communication	Efficient multiplexing	Modern low-latency transport

---

18. The Interview Answer
If the interviewer asks:
"Difference between HTTP/1.1, HTTP/2 and HTTP/3?"
Give this:
HTTP/1.1 runs over TCP and supports persistent connections, but multiple requests can suffer from head-of-line blocking. HTTP/2 still uses TCP but introduces binary framing, multiplexing, and header compression, allowing multiple streams over one connection. HTTP/3 uses QUIC over UDP. QUIC provides reliable multiplexed transport with integrated TLS and avoids TCP-level head-of-line blocking between independent streams.
That's a very strong answer.

---

19. Don't Make These Mistakes
❌ "HTTP/2 uses UDP."
No.
HTTP/2 → TCP
HTTP/3 → QUIC/UDP
❌ "HTTP/3 is unreliable."

No.
QUIC provides reliable transport semantics.
❌ "UDP is faster than TCP."
Too simplistic.
UDP has less built-in transport machinery; actual performance depends on the protocol/application.
❌ "HTTP/2 eliminates all head-of-line blocking."
No.
It eliminates/reduces HTTP-level blocking through multiplexing, but TCP's ordered byte stream can still cause transport-level HOL blocking.
❌ "HTTP/3 is just HTTP over UDP."
No.
The important middle layer is:

```
HTTP/3
  ↓
QUIC
  ↓
UDP
```

---

20. One Diagram to Memorize

HTTP EVOLUTION

```
HTTP/1.1
  |
  ▼
 TCP
  |
  ▼
 IP


HTTP/2
  |
  ├── Multiplexing
  ├── Binary framing
  └── Header compression
  |
  ▼
 TCP
  |
  ▼
 IP


HTTP/3
  |
  ▼
 QUIC
  |
  ├── Reliable transport
  ├── Multiplexed streams
  ├── TLS
  ├── Congestion control
```

└── **Connection migration**

▼

UDP

▼

IP

---

21. Quick Interview Drill

Before moving on, make sure you can answer these:

1. **Why was HTTP/2 introduced?**
2. **What is multiplexing?**
3. **HTTP/1.1 vs HTTP/2?**
4. **Does HTTP/2 use UDP?**
5. **Why does HTTP/2 still suffer from TCP head-of-line blocking?**
6. **Why does HTTP/3 use UDP?**
7. **What is QUIC?**
8. **Is HTTP/3 reliable?**
9. **How does QUIC provide reliability over UDP?**
10. **What is the major architectural difference between HTTP/2 and HTTP/3?**

The one-line memory trick:

HTTP/1.1 → TCP + basic/persistent HTTP

HTTP/2 → TCP + multiplexing

HTTP/3 → QUIC + UDP + multiplexing

Next Topic — WebSocket vs HTTP vs Long Polling vs SSE

This is a **smaller but useful SDE topic**. The key is understanding **how the communication pattern differs**, not memorizing protocol trivia.

---

1. The Core Problem

HTTP traditionally works like:

Client Server

Request ——————————>

<—————————— Response

But imagine a **chat application**.

You don't want the client constantly asking:

"Any new message?"

So we need mechanisms for **server → client updates**.

The four concepts to distinguish are:

HTTP

Long Polling

SSE

WebSocket

---

2. Normal HTTP

HTTP is fundamentally request/response.

Client Server

GET /messages ——————————>

<—————————— Response

If something new happens later, the client makes another request.

Good for

- REST APIs
- Web pages
- CRUD
- Normal request/response operations

Problem

The server cannot simply push arbitrary new data to the client through a normal completed HTTP request.

---

3. Long Polling

Long polling is a workaround using HTTP.

Client sends:

GET /messages

Server **doesn't immediately respond** if there is no new data.

Instead:

Client ——————————————> Server

waits...

New message!

Client <—————————————— Server

Then the client immediately sends another request.

Client ——————————————> Server

waits...

Client <—————————————— Server

So it creates the appearance of server push.

---

4. Why Long Polling?

Imagine a chat application.

Without long polling:

Client:

"Any message?"

"Any message?"

"Any message?"

"Any message?"

This is **short polling**.

Long polling instead says:

Client:

"Tell me when there is a message."

Server:

"Okay, I'll keep this request open."

Much better.

---

5. Server-Sent Events (SSE)

SSE is designed for:

Server → Client streaming

The client establishes an HTTP connection:

Client ——————————————> Server

connection stays open

Client <———————— Server
Client <———————— Server
Client <———————— Server
Client <———————— Server
The server can continuously send events.
Important:
SSE is **one-way**.
Server ————————→ Client
The client can still make normal HTTP requests separately, but the SSE stream itself is server-to-client.

---

6. Good Uses for SSE
Examples:

- Live notifications
- Stock price updates
- Live dashboards
- Progress updates
- News feeds
- Monitoring systems

Example:

```
Server
  |
  ├— "CPU = 50%"
  ├— "CPU = 52%"
  ├— "CPU = 49%"
  └— "CPU = 55%"
      ↓
    Browser
```

---

7. WebSocket ⭐ ⭐ ⭐ ⭐ ⭐
WebSocket provides a **persistent, bidirectional communication channel**.
After establishing the connection:
Client ⇄ Server
Both sides can send data independently.
Client ————————> Server
Client <———————— Server
Client ————————> Server
Client <———————— Server
The connection remains open.

---

8. WebSocket Handshake
WebSocket starts with an HTTP-based handshake.
Conceptually:

```
Client
  |
  | HTTP Upgrade request
  ▼
Server
  |
  | Upgrade accepted
  ▼
```

WebSocket connection
After the upgrade:
Client ⇄ Server
The communication is no longer ordinary request/response HTTP semantics.

---

9. WebSocket Use Cases
Excellent for:
Chat
Online gaming
Collaborative editing
Live trading interfaces
Real-time dashboards
Notifications
For example:
Player A
⇄
Game Server
⇄
Player B
The server can push state changes immediately.

---

10. HTTP vs Long Polling vs SSE vs WebSocket

Feature	HTTP	Long Polling	SSE	WebSocket
Basic model	Request/response	Long-held HTTP request	Server stream	Persistent connection
Server → Client	Response	Yes	Yes	Yes
Client → Server	Yes	Yes	Normal HTTP separately	Yes
Bidirectional	Request/response	Not truly persistent bidirectional	No	**Yes**
Connection	Usually short/request-based	Held temporarily	Long-lived	Long-lived
Good for	APIs/web	Simple notifications	Server updates	Real-time communication
Complexity	Low	Medium	Low/medium	Medium
Typical example	REST	Notifications	Live dashboard	Chat

---

11. The Most Important Distinction
Memorize this:
HTTP
Client asks → Server responds

Long Polling
Client asks → Server waits → responds

SSE
Server ————→ Client
continuous one-way stream

WebSocket

Client ⇄ Server
continuous two-way communication
That's enough to answer most interview questions.

---

12. Scenario: Chat Application
Interviewer:
"Would you use HTTP or WebSocket for a chat application?"
Strong answer:
I'd generally use WebSocket because chat requires low-latency bidirectional communication. Both the client and server need to be able to send messages without repeatedly establishing HTTP requests.

---

13. Scenario: Live Stock Dashboard
Interviewer:
"What about a dashboard where the server continuously sends updated prices?"
SSE could be a good choice:
Server
 |
 ├— price update
 ├— price update
 ├— price update
 └— price update
 ↓
 Client
Because the communication is primarily:
Server → Client
You don't necessarily need full WebSocket bidirectionality.

---

14. Scenario: Simple Notification System
Long polling can work when:

- Real-time requirements aren't extreme
- Infrastructure simplicity matters
- You want to stay within ordinary HTTP semantics

But for modern systems, SSE/WebSocket may be more appropriate depending on requirements.

---

15. WebSocket vs HTTP — Important Nuance
Don't say:
"WebSocket replaces HTTP."
That's wrong.
They solve different communication patterns.
A typical application might use:
HTTP
↓
REST API
↓
Login
↓
Fetch profile
↓
CRUD operations

WebSocket
↓
Real-time messages

They can coexist.

---

16. Quick Mental Model
Think of a restaurant.
HTTP
You:
"Give me today's menu."

Server:
"Here."

Conversation ends.
Long Polling
You:
"Tell me when the chef has something new."

Server:
"Okay, I'll keep your request open."

Chef prepares something.

Server:
"New dish!"
Then you ask again.
SSE
You:
"Keep me updated."

Server:
"Dish 1..."
"Dish 2..."
"Dish 3..."
One-way stream.
WebSocket
You ⇄ Chef

You: "Add spicy sauce."
Chef: "Done."
You: "What about dessert?"
Chef: "Coming up."
Two-way persistent communication.

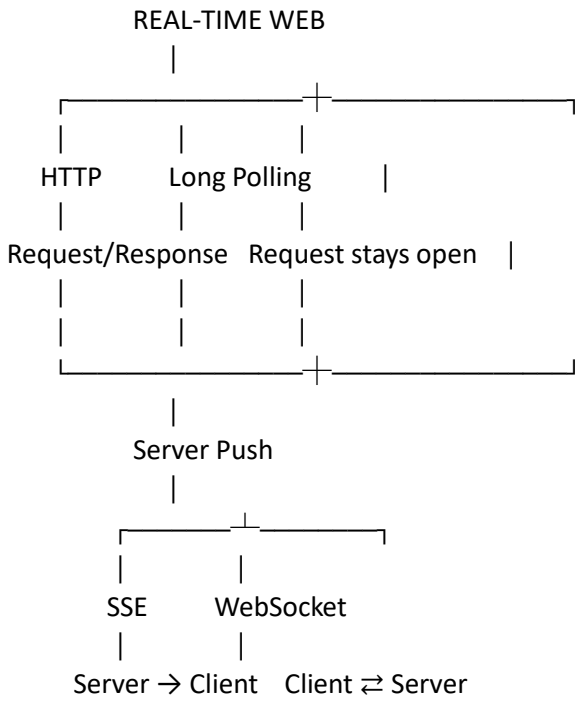
---

17. Interview Questions
Make sure you can answer these:
Q1. What is WebSocket?
A protocol providing persistent, bidirectional communication between client and server.
Q2. WebSocket vs HTTP?
HTTP is primarily request/response, while WebSocket provides a persistent bidirectional channel.
Q3. What is SSE?
Server-Sent Events provide a persistent HTTP-based server-to-client event stream.
Q4. SSE vs WebSocket?
SSE → Server → Client
WebSocket → Client ⇄ Server

Q5. Why use WebSocket for chat?
Because chat requires low-latency bidirectional communication.
Q6. Why not WebSocket for everything?
Because you may not need persistent bidirectional communication. Ordinary HTTP is simpler for normal APIs, and SSE can be simpler when communication is only server → client.
Q7. What is long polling?
The client sends an HTTP request and the server keeps it open until data is available or a timeout occurs, after which the client makes another request.

---

18. One Diagram to Memorize



---

19. HPE Interview Priority
For your interview:
WebSocket ⭐⭐⭐⭐⭐
HTTP vs WebSocket ⭐⭐⭐⭐⭐
SSE ⭐⭐⭐
Long Polling ⭐⭐⭐
Detailed WebSocket frames ⭐
Don't waste time learning the WebSocket frame format, masking algorithm, opcodes, etc. unless the interviewer specifically goes there.

Today's takeaway
If the interviewer gives you a system and asks **"Which communication mechanism would you choose?"**, reason from the direction of communication:
Normal API
↓
HTTP

Server → Client updates
↓
SSE

Simple delayed HTTP notification
↓

Long Polling

Real-time Client ⇄ Server
↓
WebSocket

Next Topic — Proxy, Reverse Proxy, Load Balancer & CDN
This is the **last major modern-web networking cluster** I want you to know for the HPE SDE interview.
These four are easy to confuse because all of them sit **between clients and servers**, but their purposes are different.

---

1. The Basic Idea
Without intermediaries:
Client
|
▼
Server
In production:
Client
|
▼
?
|
▼
Server
That ? could be:

- Proxy
- Reverse proxy
- Load balancer
- CDN
- API Gateway
- Firewall/WAF

The important question is:
Who is the intermediary representing?

---

2. Proxy ⭐ ⭐ ⭐ ⭐
A **forward proxy** sits on the **client side**.
Client
|
▼
Proxy
|
▼
Internet
|
▼
Server
The server sees the proxy as the source rather than directly interacting with the client in the same way.

Why use a Forward Proxy?
Common purposes:
1. Privacy / identity masking
Client

↓
Proxy
↓
Internet

2. Access control

An organization can control what employees can access.

Employee
↓
Corporate Proxy
↓
Internet

3. Caching

The proxy may cache frequently requested resources.

4. Filtering

For example:

Employee → Proxy → Website
|
└— Blocked

---

3. Reverse Proxy ⭐⭐⭐⭐⭐

Now reverse the perspective.

A reverse proxy sits in front of **servers**.

Client
|
▼
Reverse Proxy
|
├——— Server A
├——— Server B
└——— Server C

The client doesn't necessarily know which backend server actually handled the request.

---

4. Why Use a Reverse Proxy?

Several reasons.

Security

Backend servers don't need to be directly exposed to the Internet.

Internet
↓
Reverse Proxy
↓
Private Backend

TLS termination

The reverse proxy can handle HTTPS:

Client
| HTTPS
▼
Reverse Proxy
| HTTP/internal TLS
▼
Application

Caching

It can cache responses.

Routing

It can route requests based on:
/path
Host
headers
cookies
Example:
/api/* → API server
/images/* → Image server
/admin/* → Admin server

---

5. Forward Proxy vs Reverse Proxy ⭐⭐⭐⭐⭐
This is a very common interview question.

Forward Proxy	Reverse Proxy
Represents clients	Represents servers
Client knows about it	Client may not know about it
Outbound traffic	Inbound traffic
Controls client access	Protects/routes backend
Client → Proxy → Internet	Client → Proxy → Server

Memory trick:
Forward Proxy
↓
FORWARD client's requests

Reverse Proxy
↓
SITS IN FRONT OF servers

---

6. Load Balancer ⭐⭐⭐⭐⭐
A load balancer distributes incoming traffic across multiple servers.
Instead of:
Client
|
▼
Server
we have:

```
        ┌── Server A
        |
Client → Load Balancer ── Server B
        |
        └── Server C
```

Why?
Scalability and availability.

---

7. Why Do We Need Load Balancing?
Suppose one server can handle:
1000 requests/sec
But your application receives:
5000 requests/sec
One server isn't enough.
So:

```
        ┌── 1000
        │
Requests → LB ├── 1000
        │
        ├── 1000
        │
        ├── 1000
        │
        └── 1000
```

Now we can handle much more traffic.

---

8. Load Balancing Algorithms

For your interview, know the basic ones.

Round Robin

Send requests sequentially:

A → Server 1
B → Server 2
C → Server 3
D → Server 1
E → Server 2

Simple.

Weighted Round Robin

Servers have different capacities.

Server A → weight 3
Server B → weight 1

A gets roughly three times as much traffic.

Least Connections

Send traffic to the server with the fewest active connections.

Server A → 10 connections
Server B → 4 connections
Server C → 7 connections

Next request → B

IP Hash

Use the client's IP to choose a server.

Useful when you want a degree of session affinity.

```
Client IP
  ↓
Hash
  ↓
Server
```

---

9. Health Checks ⭐ ⭐ ⭐ ⭐

A load balancer shouldn't send traffic to a dead server.

It periodically checks backend health.

```
Load Balancer
  |
  ├── Server A → Healthy
  ├── Server B → Healthy
```

└── Server C → DOWN

Then:

Traffic

↓

A / B

and C is removed from rotation.

---

10. Load Balancer Types

At a high level:

Layer 4 Load Balancer

Works using transport information.

For example:

IP

Port

TCP

UDP

Conceptually:

Client

↓

L4 Load Balancer

↓

Backend

It doesn't necessarily need to understand HTTP.

Layer 7 Load Balancer

Understands application-layer information.

For example:

HTTP path

Host

Headers

Cookies

So it can do:

/api

↓

API servers

/static

↓

Static servers

This distinction is useful in SDE/system-design interviews.

---

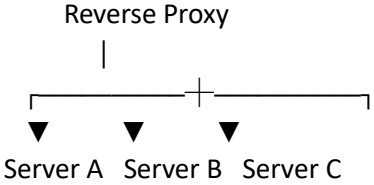
11. Reverse Proxy vs Load Balancer

Here's where people get confused.

A reverse proxy and load balancer **can overlap**.

A reverse proxy can perform load balancing.

For example:

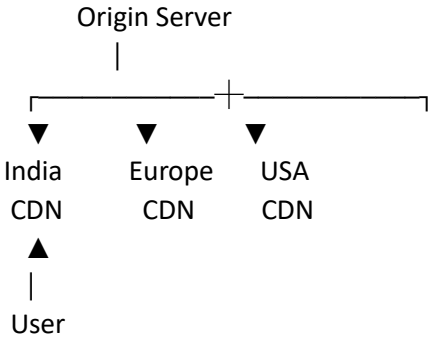


The distinction is primarily about **role/function**, not necessarily separate physical machines.

A reverse proxy is an intermediary in front of servers; load balancing is the function of distributing traffic among backend servers.
A single component can do both.

---

12. CDN ⭐⭐⭐⭐⭐
CDN = **Content Delivery Network**
The basic idea:
Put content closer to users.
Without CDN:
India User
  |
  | long distance
  ▼
US Server
With CDN:



---

13. What Does a CDN Cache?
Typically:

- Images
- CSS
- JavaScript
- Videos
- Fonts
- Static files

Sometimes APIs or dynamic content can also be cached depending on configuration.

---

14. CDN Cache Hit
Suppose:
User requests:
logo.png
CDN already has it:
User
↓
CDN
↓
logo.png
That's a:
Cache hit
The CDN doesn't need to contact the origin.

---

15. CDN Cache Miss
If the CDN doesn't have it:
User
↓

CDN
↓
Origin Server
↓
CDN caches response
↓
User
Next user may get it directly from the CDN.
User 1 → CDN → Origin
↓
Cache

User 2 → CDN → Cached content

---

16. Why CDN Improves Performance
Main reason:
Reduce network distance and latency.
Instead of:
User → 10,000 km → Origin
you may have:
User → nearby CDN
Benefits:

- Lower latency
- Reduced origin load
- Better scalability
- Faster static content delivery

---

17. CDN vs Load Balancer
Important distinction.
Load Balancer
Distributes requests across backend servers.
Load Balancer
/ | \
A B C
CDN
Distributes/caches content across geographically distributed edge locations.
Origin
/ | \
CDN CDN CDN
So:
Load Balancer → distribute compute

CDN → distribute/cache content geographically

---

18. CDN vs Reverse Proxy
These can also overlap.
A CDN edge server can act like a reverse proxy:
Client
↓
CDN Edge
↓
Origin
But the CDN adds a major capability:

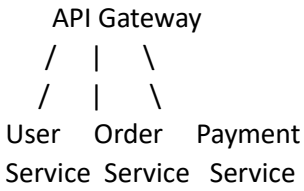
Globally distributed caching and edge delivery.

---

19. API Gateway ⭐⭐⭐⭐

You may hear this in an SDE interview.

An API Gateway is an entry point for APIs, especially in microservices.



It can handle:

- Routing
- Authentication
- Rate limiting
- Logging
- Request transformation
- TLS termination
- Service aggregation

---

20. Reverse Proxy vs API Gateway

They overlap too.

A reverse proxy is a broader networking intermediary.

An API gateway is generally **application/API-focused** and often provides additional API-management functionality.

Think:

Reverse Proxy
↓
General traffic intermediary

API Gateway
↓
API-aware gateway with policies/features

---

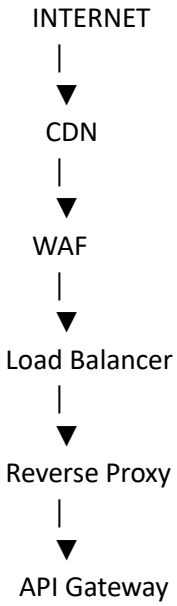
21. Full Production Architecture

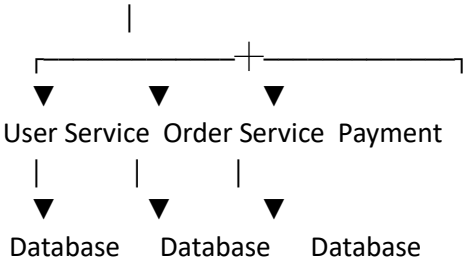
Now put everything together.

Imagine you open:

https://example.com/api/users

A production architecture might look like:





Not every system needs all of these.

That's important.

Don't blindly put CDN + WAF + LB + reverse proxy + API gateway into every architecture.

Each exists for a reason.

---

22. Interview Scenario

Question:

"We have one backend server and traffic is increasing. What would you do?"

Weak answer:

"Add a load balancer."

That's incomplete.

Better reasoning:

Traffic increasing

↓

Can one server handle it?

↓

No

↓

Add multiple instances

↓

Load balancer

↓

Health checks

↓

Horizontal scaling

---

23. Another Scenario

"Users in India complain that images load slowly, but users near the server are fast."

Think:

Origin is geographically distant

↓

High latency

↓

CDN

↓

Cache images near users

---

24. Another Scenario

"We don't want users to directly access our backend servers."

Think:

Internet

↓

Reverse Proxy / Load Balancer

↓

Private Backend Servers

The backend servers can live on a private network.

25. Another Scenario

"We have /api/* and /static/* and want them handled by different backends."

A Layer-7 reverse proxy can inspect the HTTP path:

/api/*
  ↓
API Server

/static/*
  ↓
Static Server

---

26. The Four Things to Memorize

This is the most important part.

FORWARD PROXY
Client → Proxy → Internet
        ↑
    Represents client

REVERSE PROXY
Client → Proxy → Backend
        ↑
    Represents servers

LOAD BALANCER
Client → LB → Server A/B/C
        ↑
    Distributes traffic

CDN
Client → Nearby Edge → Content
        ↑
    Caches/distributes content

---

27. Comparison Table

Component	Main Purpose	Represents	Key Benefit
Forward Proxy	Client-side intermediary	Client	Access control/privacy
Reverse Proxy	Server-side intermediary	Server/backend	Security/routing
Load Balancer	Distribute traffic	Backend pool	Scalability/availability
CDN	Cache content near users	Content/origin	Lower latency
API Gateway	Manage APIs	API services	Routing/auth/rate limiting

---

28. HPE Interview Questions

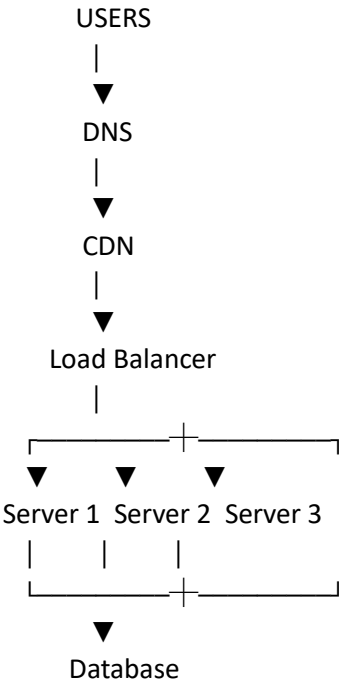
Be ready for:

Q1
What is a proxy?
Q2
Forward proxy vs reverse proxy?

Q3
What is a load balancer?
Q4
Why do we need load balancing?
Q5
What is round-robin load balancing?
Q6
What happens if one backend server dies?
Health checks remove it from the available pool.
Q7
What is a CDN?
Q8
CDN vs load balancer?
Q9
Can a reverse proxy act as a load balancer?
Yes.
Q10
What is an API Gateway?
Q11
Why shouldn't backend servers always be directly exposed?
Security, controlled access, centralized TLS/authentication, routing, and reduced attack surface.

---

29. The Architecture You Should Be Able to Draw
If they ask:
"Design a basic scalable web architecture."
You should be comfortable drawing:



And explain **why each component exists**.

Final mental model
WHO ARE YOU REPRESENTING?

Forward Proxy
→ Client

Reverse Proxy

→ Server

WHAT ARE YOU DOING?

Load Balancer
→ Distributing requests

CDN
→ Bringing cached content closer to users
That's enough depth for this topic.

Absolutely. **We'll leave CN + DSA for later** and finish the remaining **theory + scenario-based networking** now.
At this point, the core is covered. What remains is mostly **quick-hit topics that can appear as follow-ups**, followed by the scenarios that actually test whether you understand CN.

Part 1 — Remaining CN Topics

1. IPv6 ⭐ ⭐ ⭐

IPv4 has a 32-bit address space:
192.168.1.10
IPv6 uses **128 bits**:
2001:0db8:85a3:0000:0000:8a2e:0370:7334
So IPv6 provides an enormously larger address space.

Why IPv6?

Primarily:
IPv4 address exhaustion.
IPv6 also provides improvements such as:

- Simplified header structure
- Better support for autoconfiguration
- No broadcast
- Multicast/anycast mechanisms

IPv4 vs IPv6

IPv4	IPv6
32-bit	128-bit
192.168.1.1	2001:db8::1
Broadcast exists	No broadcast
ARP	NDP
NAT widely used	NAT generally not required for address conservation

Interview answer

IPv4 uses 32-bit addresses while IPv6 uses 128-bit addresses, primarily solving the address-exhaustion problem and providing a much larger address space.
That's enough for your interview.

---

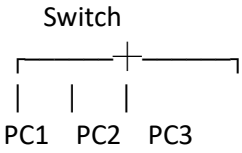
2. NDP — Neighbor Discovery Protocol

In IPv4:
IP → MAC
is commonly handled through:
ARP
IPv6 uses:
Neighbor Discovery Protocol (NDP)
which operates using ICMPv6.
So remember:

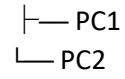
IPv4 → ARP
IPv6 → NDP
Don't spend time memorizing individual NDP message types unless asked.

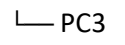
---

3. VLAN ⭐ ⭐ ⭐
VLAN = **Virtual Local Area Network**
It allows a physical switch network to be divided into multiple logical networks.
Imagine:



Without VLANs, they're potentially in the same Layer-2 broadcast domain.
With VLANs:





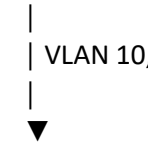
Now VLAN 10 and VLAN 20 are separate Layer-2 broadcast domains.
Communication between VLANs generally requires Layer-3 routing.
Interview answer
A VLAN logically separates devices into different Layer-2 broadcast domains even when they share the same physical switching infrastructure.

---

4. Access Port vs Trunk Port
A small but useful VLAN follow-up.
Access port
Normally carries traffic for **one VLAN** toward an endpoint.



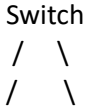
Trunk port
Carries traffic for **multiple VLANs**, typically between network devices.



You don't need deep 802.1Q frame-format knowledge.

---

5. STP ⭐ ⭐ ⭐
STP = **Spanning Tree Protocol**
Why is it needed?
Suppose switches form a loop:



Switch B —— Switch C

Layer-2 loops can cause problems such as:

- Broadcast storms
- Duplicate frames
- MAC-table instability

STP logically blocks some redundant paths:

```
    Switch A
     /    \
    /      \
 Switch B    Switch C
    \      /
     \--X--/
      blocked
```

If a path fails, STP can potentially activate a redundant path.

Interview answer

STP prevents Layer-2 switching loops by logically blocking redundant paths while retaining them for failover.

---

6. VPN ⭐ ⭐ ⭐

VPN = **Virtual Private Network**

It creates a protected tunnel over an untrusted network.

Conceptually:

```
Office A
  |
  | encrypted tunnel
  ▼
Internet
  |
  ▼
Office B
```

Common purposes:

- Secure remote access
- Site-to-site connectivity
- Protecting traffic over untrusted networks

Important distinction:

A VPN does not magically make you anonymous or secure every aspect of your online activity.

For interview purposes:

Encrypted/protected tunnel over an untrusted network.

---

7. Bandwidth vs Throughput ⭐ ⭐ ⭐ ⭐

Classic interview trap.

Bandwidth

Maximum theoretical capacity.

Example:

1 Gbps link

Throughput

Actual rate of useful data transferred.

You may have:

Bandwidth = 1 Gbps

Throughput = 700 Mbps

because of:

- Congestion
- Protocol overhead
- Packet loss

- Hardware limitations
- Network conditions

Memory trick

Bandwidth → capacity

Throughput → actual achieved rate

---

8. Latency vs RTT ⭐ ⭐ ⭐ ⭐

Latency

Time required for data to travel between endpoints, depending on what exactly is being measured.

RTT

Round-Trip Time

Time:

Client

↓

Server

↓

Client

For example:

Request → Server → Response

= 50 ms RTT

Ping commonly gives you an RTT measurement.

---

9. Jitter ⭐ ⭐ ⭐

Jitter is variation in packet delay.

Suppose packets arrive:

10 ms

11 ms

10 ms

50 ms

12 ms

The variation is problematic for real-time applications.

Especially relevant to:

- VoIP
- Video conferencing
- Online gaming

---

10. Packet Loss ⭐ ⭐ ⭐

Packet loss means packets fail to reach their destination.

Possible causes:

- Congestion
- Faulty hardware
- Wireless interference
- Routing problems
- Network failures

TCP can retransmit lost data.

UDP itself doesn't provide TCP-style retransmission.

---

11. Propagation vs Transmission Delay

You don't need deep formulas, but understand the distinction.

Transmission delay

Time required to **put the bits onto the link**.

Affected by:

Packet size
Link bandwidth
Propagation delay
Time for the signal to **travel through the medium**.
Affected by:
Distance
Signal propagation speed
Easy distinction:
Transmission → putting bits onto wire

Propagation → bits traveling through wire

---

12. Firewall ⭐⭐⭐⭐
A firewall controls network traffic according to rules.
Example:
Allow:
TCP 443

Block:
TCP 23
It can filter based on things such as:

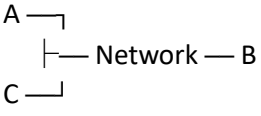
- Source IP
- Destination IP
- Port
- Protocol
- Connection state
- Application information, depending on firewall type

Simple architecture
Internet

Firewall

Internal Network
Don't confuse:
Firewall → controls traffic
Router → forwards traffic
A device can perform both functions, but the concepts are different.

---

13. Packet Switching vs Circuit Switching ⭐⭐
Circuit switching
Dedicated communication path.
Traditional telephone systems are the classic example.
A ═══════ B
Packet switching
Data is divided into packets that share network resources.

The Internet uses **packet switching**.
For your interview, that's sufficient.

---

14. MTU ⭐ ⭐ ⭐

MTU = **Maximum Transmission Unit**

Maximum packet/frame payload size that can be transmitted over a particular link without fragmentation at that layer.

A common Ethernet MTU is:

1500 bytes

Don't blindly say:

"MTU is always 1500."

It isn't.

Different networks can have different MTUs.

---

15. Quick Topic: Symmetric vs Asymmetric Encryption

This is technically more security/cryptography than core CN, but it can appear when discussing HTTPS.

Symmetric

Same key:

Key → Encrypt

Key → Decrypt

Fast.

Asymmetric

Key pair:

Public key

Private key

Useful for authentication/key exchange.

HTTPS/TLS uses **both types of cryptography** for different purposes.

Don't say:

"HTTPS encrypts everything using RSA."

Modern TLS commonly uses asymmetric cryptography for authentication/key establishment and symmetric cryptography for bulk data encryption.

Part 2 — Scenario-Based CN

Now we get to the **important part**.

These are the questions I'd much rather have you practice than memorize obscure protocols.

Scenario 1 — "What happens when you type google.com?"

You should now be able to give a layered answer.

Browser

↓

DNS

↓

IP address

↓

Routing

↓

ARP / Gateway

↓

TCP or QUIC

↓

TLS

↓

HTTP

↓

Server

For HTTPS over HTTP/2:

DNS
↓
TCP
↓
TLS
↓
HTTP/2
For HTTP/3:
DNS
↓
QUIC
↓
HTTP/3

Interview answer

First the browser resolves the domain using DNS. It determines how to reach the destination IP through the routing table and, if necessary, uses ARP to reach the local gateway. For HTTPS over HTTP/1.1 or HTTP/2, a TCP connection is established followed by TLS, and then the browser sends an HTTP request. The server processes it and returns the response. With HTTP/3, QUIC over UDP replaces TCP.

That's an excellent answer.

Scenario 2 — "Internet isn't working. How do you troubleshoot?"

Don't immediately say:

"Check DNS."

Start from lower layers.

1. Physical connectivity
   ↓
2. Network interface
   ↓
3. IP configuration
   ↓
4. Default gateway
   ↓
5. Routing
   ↓
6. DNS
   ↓
7. TCP port
   ↓
8. TLS
   ↓
9. HTTP/application

Useful Linux commands:

ip addr
ip route
ip neigh
ping 192.168.1.1
ping 8.8.8.8
nslookup google.com
dig google.com
curl -v https://google.com

Strong interview answer

I'd isolate the failure layer by layer. First I'd verify the interface and IP configuration, then connectivity to the default gateway, then external IP connectivity, then DNS resolution, then TCP connectivity to the required port, and finally TLS and HTTP/application behavior.
This is exactly the kind of structured problem-solving answer interviewers like.

Scenario 3 — Ping IP works, domain doesn't
Given:
ping 8.8.8.8
works.
But:
ping google.com
fails.
Likely suspect:
DNS.
Test:
nslookup google.com
or:
dig google.com
The reasoning:
IP connectivity works
↓
Network probably works
↓
Name resolution fails
↓
Investigate DNS

Scenario 4 — DNS works, but HTTPS doesn't
Suppose:
nslookup example.com
works.
But:
curl https://example.com
fails.
Now investigate:
DNS
↓
works
↓
TCP 443
↓
TLS
↓
HTTP
Potential issues:

- Firewall
- Port 443 inaccessible
- Server not listening
- TLS certificate/configuration problem
- Reverse proxy failure
- Application failure

Scenario 5 — Port 80 works, 443 doesn't

Suppose:
HTTP → works
HTTPS → fails
That immediately narrows things down.
Likely areas:
TCP 80 → working
TCP 443 → investigate

Firewall
TLS
Certificate
HTTPS listener
Reverse proxy
You don't need to investigate DNS first because DNS is already clearly working if you're reaching the host.

Scenario 6 — Two computers on the same LAN can't communicate
Suppose:
PC A
192.168.1.10

PC B
192.168.1.20
Both should be on:
192.168.1.0/24
Check:
Interface
↓
IP address
↓
Subnet
↓
ARP
↓
Switch
↓
Firewall
Useful:
ip addr
ip neigh
ping 192.168.1.20
If ARP isn't resolving:
192.168.1.20
↓
ARP ?
↓
No MAC
Investigate the Layer-2 path/configuration.

Scenario 7 — Server is running, but nobody can connect
Classic interview question.
You know:
Application process → running
But that doesn't mean it is reachable.

Check:
Process
↓
Listening socket
↓
Correct IP
↓
Correct port
↓
Firewall
↓
Routing
↓
Load balancer / reverse proxy
Linux:
ss -lntp
Then test:
nc -vz <server> <port>

Scenario 8 — Users complain the website is slow
Don't immediately blame the server.
Break it down:
DNS latency
↓
Network RTT
↓
TCP/QUIC establishment
↓
TLS
↓
Server processing
↓
Database
↓
Response size
↓
Client rendering
If static content is slow globally:
CDN might help.
If backend processing is slow:
Scaling, caching, database optimization, etc.
If only distant users are slow:
Latency/CDN/geographic distribution.

Scenario 9 — One Backend Server Dies
Architecture:

```
        Load Balancer
        /       \
        A         B
```

Server A dies.
A good load balancer:
Health check
↓

A unhealthy
↓
Remove A
↓
Send traffic to B
This is why health checks matter.

Scenario 10 — Traffic suddenly increases 10×
Suppose:
100 requests/sec
↓
1000 requests/sec
Possible response:
Load Balancer
↓
Horizontal scaling
↓
Multiple instances
Then consider:
Caching
CDN
Database scaling
Rate limiting
Queueing
For a basic CN answer:
Use multiple backend instances behind a load balancer and scale horizontally.

Scenario 11 — Users in India are slow, US users are fast
Suppose origin is in the US.
India ——————→ US
Large geographical distance means higher network latency.
A CDN can provide:
India
↓
Nearby CDN Edge
↓
Cached content
So:
CDN is the obvious networking-level optimization for static/cacheable content.

Scenario 12 — Client can connect to IP but not domain
Again:
IP works
Domain fails
Think:
DNS.
But don't blindly conclude it.
You should verify:
dig example.com
If DNS works, then the issue could be application-level or hostname-specific routing/configuration.
Good troubleshooting means testing your hypothesis, not declaring it.

Scenario 13 — Why Can You Ping a Server But Can't Open Its Website?

Excellent interview question.
Possible explanation:
ping
↓
ICMP
while website:
HTTPS
↓
TCP 443
↓
TLS
↓
HTTP
Therefore:
ICMP working does not prove TCP 443 or HTTPS is working.
Potential causes:

- Port 443 blocked
- Web server down
- Firewall
- TLS problem
- Reverse proxy issue
- Application failure

Scenario 14 — Why Does Ping Fail But Website Works?
Also possible.
The server/firewall may block:
ICMP
while allowing:
TCP 443
So:
ICMP ❌
HTTPS ✅
is perfectly possible.
This is a **very good interview concept** because it tests whether you understand that ping and HTTP use different protocols.

Scenario 15 — What Happens When a TCP Packet Is Lost?
Suppose:
Sender:

1
2
3
4
5
Packet 3 gets lost.
TCP uses sequence numbers and acknowledgements.
The receiver can indicate what data it has received.
The sender can detect missing data and retransmit.
Conceptually:
1 → ACK
2 → ACK

3 → X
4 → ACK
5 → ACK

Sender detects missing data
↓
Retransmission
↓
3
The exact retransmission mechanism can involve duplicate ACKs and retransmission timers.
For your interview:
TCP detects loss using acknowledgements and timers/duplicate ACK behavior and retransmits the missing data.

Scenario 16 — Flow Control vs Congestion Control
Interviewer:
"TCP is sending too fast. Who are you protecting?"
Receiver overwhelmed?
Flow Control
Network overloaded?
Congestion Control
Memorize:
Receiver → Flow
Network → Congestion

Scenario 17 — Why Does a Router Need a MAC Address If IP Already Exists?
Excellent conceptual question.
Because IP and MAC operate at different scopes.
IP
↓
End-to-end logical addressing/routing

MAC
↓
Local link delivery
For example:
Laptop
|
| Destination IP = Server
| Destination MAC = Gateway
▼
Router
At the next hop, a new Layer-2 frame is created.
So:
IP helps determine where the packet ultimately goes; MAC addresses are used for delivery across the current local link.

Scenario 18 — Why Can't My Private IP Be Used Directly on the Internet?
Because private IPv4 addresses are not globally routable.
Example:
192.168.1.10
Instead:
Laptop
192.168.1.10

↓
NAT
↓
Public IP
↓
Internet

Scenario 19 — What Happens If DNS Server Is Down?
Potentially:
DNS resolution fails
↓
Domain names can't be resolved
But existing connections may continue.
And cached DNS records may still work until their TTL expires.
This is an important nuance.
DNS failure does **not necessarily mean every existing network connection immediately breaks**.

Scenario 20 — TCP vs UDP Scenario
Interviewer:
"Why might DNS use UDP?"
Answer:
DNS queries are often small request/response exchanges where the lower overhead and lack of connection establishment are useful. DNS can also use TCP when required.
Then they might ask:
"Why would a video call use UDP?"
Answer:
Real-time applications often prioritize timeliness over retransmitting every lost packet. TCP's ordering and retransmission behavior can introduce delay, while applications can tolerate some packet loss depending on the design.
Don't say:
"UDP guarantees low latency."
It doesn't.

The Final CN Scenario Framework
When given **any networking problem**, think in this order:

APPLICATION
|
HTTP / DNS
|
TLS
|
TCP / UDP / QUIC
|
IP
|
ARP / MAC / VLAN
|
SWITCH
|
ROUTER
|
INTERNET

Then ask:

1. Is the machine connected?
2. Does it have an IP?
3. Is the destination local or remote?
4. Can it reach the gateway?
5. Can it reach the destination IP?
6. Does DNS resolve?
7. Is the required port reachable?
8. Is TLS working?
9. Is the HTTP/application layer working?

That gives you a **systematic troubleshooting methodology**, rather than guessing.

CN Theory Status

At this point, for your **General SDE/HPE target**, I'd consider the CN syllabus essentially complete:

Core

- OSI
- TCP/IP
- Encapsulation
- TCP
- UDP
- Handshake
- Termination
- Flow/congestion control
- IP addressing
- Subnetting/CIDR
- Routing
- NAT
- MAC
- ARP
- Switching
- DHCP
- ICMP
- DNS
- HTTP/HTTPS
- FTP
- SMTP/IMAP/POP3
- Email flow

Modern Web

- HTTP/1.1
- HTTP/2
- HTTP/3
- QUIC
- WebSocket
- SSE
- Long Polling
- Proxy
- Reverse Proxy
- Load Balancer
- CDN
- API Gateway

Remaining supporting concepts

- IPv6
- NDP
- VLAN

- STP
- VPN
- Firewall
- Bandwidth/throughput
- Latency/RTT
- Jitter
- Packet loss
- MTU
- Transmission/propagation delay
